

Spring vs Spring Boot

Spring is a framework.

Spring Boot is Spring with intelligence, automation, and defaults.

Spring Boot did not replace Spring.

It removed **configuration pain** from Spring.

What is Spring Framework ??

Spring is an **enterprise application framework** built around two core principles:

- Inversion of Control (IoC)
- Dependency Injection (DI)

Without Spring:

```
class OrderController {  
    OrderService service = new OrderService();  
}
```

This creates:

- Tight coupling
- Hard to test
- Hard to replace

With Spring:

```
@Component  
class OrderService {}  
  
@RestController  
class OrderController {  
    @Autowired  
    OrderService service;  
}
```

“How Spring internally works around DI + IoC”

ApplicationContext

- scans beans
- creates objects
- wires dependencies
- manages lifecycle

This gives:

- ✓ Loose coupling
- ✓ Easy mocking
- ✓ Replaceable implementations

This is why Spring became the standard for enterprise Java.

What problems Spring solved ??

Before Spring:

- EJB
- Heavy servers
- Huge XML
- Tight coupling
- No unit testing

Spring fixed this by:

- Using POJOs
- Using DI
- Using annotations
- Supporting testing

But Spring didn't stop there.

It built powerful modules **on top of DI**:

Module	What it does
Spring JDBC	Simplifies database access
Spring ORM	Integrates Hibernate & JPA
Spring MVC	Web & REST APIs

Spring AOP	Logging, transactions, security
Spring JMS	Messaging
Spring Test	Testing

Why Spring JDBC exists ??

Traditional JDBC:

```
Connection con = DriverManager.getConnection(...);
PreparedStatement ps = con.prepareStatement(...);
ResultSet rs = ps.executeQuery();
// loop
rs.close();
ps.close();
con.close();
```

Spring JDBC:

```
jdbcTemplate.query("select * from users", rowMapper);
```

Same database.

Less plumbing.

More testable.

Spring modules don't add new features.

They remove **boilerplate and coupling**.

Why Spring apps became painful ??

To build a Spring web app you needed:

- DispatcherServlet
- ComponentScan
- ViewResolver
- Web.xml
- DataSource
- Hibernate
- TransactionManager

A simple REST API needed 100+ lines of config.

This is where **Spring Boot** enters.

What is Spring Boot ??

Spring Boot is **Spring + Auto Configuration + Embedded Server + Starters**

Spring Boot focuses on:

- Speed
- Convention
- Automation

You write:

```
@SpringBootApplication  
public class App {}
```

Boot:

- Creates ApplicationContext
- Scans beans
- Configures MVC
- Configures Tomcat
- Configures JSON
- Configures JPA
- Starts server

You write zero XML.

What is Auto-Configuration ??

Spring Boot checks:

“What jars are on the classpath?”

If it sees:

- spring-web → configure DispatcherServlet
- spring-data-jpa → configure Hibernate
- jackson → configure JSON
- tomcat → configure server

This happens because of:

@EnableAutoConfiguration

You can override anything.

Boot just gives **smart defaults**.

Example: DataSource auto-config

Without Boot:

```
@Bean
public DataSource ds() {
    return new HikariDataSource(...);
}
```

With Boot:

Add this in application.properties:

```
spring.datasource.url=jdbc:mysql://localhost/db
spring.datasource.username=root
spring.datasource.password=pass
```

Boot creates DataSource automatically.

What are Starters ??

Instead of adding 10 dependencies, you add one: **spring-boot-starter-web**

It pulls:

- Spring MVC
- Jackson
- Tomcat
- Validation

And auto-configures everything.

Spring vs Spring Boot

Spring	Spring Boot
Framework	Framework + Automation
Needs XML & beans	Zero config
External server	Embedded server
Manual wiring	Auto-configuration
Slow setup	Fast startup

Interview Killer Line

“Spring Boot is Spring with opinionated defaults that remove infrastructure setup.”

Spring solved:

- Object creation
- Dependency management
- Enterprise architecture

Spring Boot solved:

- Configuration hell
- Startup complexity
- Deployment friction

Spring is power.

Spring Boot is speed.



Why Every **Java** Developer Should Learn **Spring Framework**!



Today, I want to share my perspective on the **Spring Framework** and why it's a game changer in **Java** development:

1 Why was Spring Framework introduced?

To **simplify Java EE development**, replacing heavy EJBs and reducing boilerplate code.

2 Problems before Spring

- Heavy XML configuration
- Tight coupling & difficult testing
- Hard to scale & maintain

3 What is Spring?

A lightweight, open-source Java framework using **Dependency Injection (DI)** to make development:



Flexible



Loosely coupled

Easier to test

4 How does Spring work?

Spring uses an **IoC Container** to manage objects:

- Automatically creates them
- Injects dependencies
- Manages their lifecycle



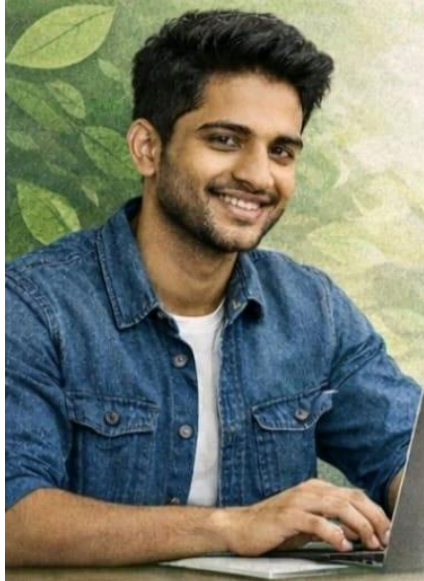
5 Advantages of Spring

- ✓ Reduces boilerplate code
- ✓ Supports modularity & loose coupling
- ✓ Eases testing
- ✓ Modern (Microservices, REST APIs, Cloud)



Final Thought

Spring is more than just a framework - it's an ecosystem enabling efficient, production-ready Java apps. **Learning Spring**



Why Java Developers Started Loving Spring Boot



Spring Boot

#1 Choice for Enterprise & Microservices

✓ Auto-Configuration

Beans auto-configured based on the classpath



✓ Embedded Server

Just run your app as a .jar file with built-in Tomcat



✓ Starter Dependencies

Pre-built starter packs for databases, security, and more



✓ Production-Ready Features

Actuator provides health checks and metrics



Convention Over Configuration

Focus on Business Logic, not Infrastructure

Technical Deep Dive: The Layered Architecture

Decoupled, Testable, and Scalable Code



Summary

How Spring Actually Works Behind the Scenes (The Engine You Don't See)

Spring is not magic.

It is an **object factory with a brain**.

When a Spring application starts, the first thing that comes alive is not your Controller, not your Service, not your Repository — it is the **Spring Container (ApplicationContext)**.

This container does four critical things:

1. It scans your application for classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`
2. It creates objects for those classes
3. It figures out how those objects depend on each other
4. It wires them together into a complete object graph

That means your application is not a random collection of Java objects — it is a **managed dependency graph** created and owned by Spring.

Without Spring, object creation looks like this:

```
OrderService service = new OrderService();
```

```
PaymentService payment = new PaymentService();
```

```
service.setPaymentService(payment);
```

Every class decides what it depends on.

Every class creates its own collaborators.

This is **tight coupling**.

With Spring, object creation is inverted:

@Service

```
class OrderService {  
    @Autowired  
    PaymentService paymentService;  
}
```

Now:

- OrderService does NOT decide what PaymentService is
- Spring decides
- Spring creates
- Spring injects

This is **Inversion of Control**.

Dependency Injection is simply **how IoC is implemented**.

The result is:

- Classes depend on **abstractions**
- Object wiring happens **outside the class**
- Code becomes **testable**
- Code becomes **replaceable**
- Architecture becomes **flexible**

This is why Spring is an enterprise framework — not because of annotations, but because of **control over object graphs**.

Spring Was Not Built to Replace Technologies — It Was Built to Orchestrate Them

Spring never tried to replace databases, ORMs, web servers, or testing frameworks.

It did something far more powerful.

It became the **glue layer** between them.

Hibernate is great at ORM.

JUnit is great at testing.

Tomcat is great at serving HTTP.

JDBC is great at talking to databases.

Spring does not compete with them — it **integrates them**.

Spring uses:

- Hibernate for ORM
- JUnit + Mockito for testing
- Tomcat for HTTP
- JDBC for database access

But Spring **controls how they are wired together** using Dependency Injection.

This is why Spring became dominant in enterprise Java — not because it reinvented technology, but because it **orchestrated complexity**.

Why Spring MVC Exists

Before Spring MVC, Java web apps were built using Servlets and JSPs.

Everything was tied together:

- URL parsing
- Request handling
- Business logic
- View rendering

Spring MVC introduced:

- DispatcherServlet (front controller)
- Handler mapping
- Controller classes

- View resolvers

This separated concerns:

Request → Controller → Service → View

This decoupling made large web applications manageable.

Why Spring Applications Became Painful to Build

As powerful as Spring was, it had a serious problem:

Configuration Explosion

A real Spring application required:

- DataSource
- EntityManagerFactory
- TransactionManager
- ComponentScan
- DispatcherServlet
- ViewResolver
- Jackson converters
- Security filters
- And dozens of XML or Java configs

Every project started with **hundreds of lines of infrastructure wiring** before a single API was written.

Spring solved architecture — but not developer experience.

That is where Spring Boot enters.

What Spring Boot Actually Does

Spring Boot did not change Spring's core.

It changed how Spring is **configured**.

Spring Boot looks at:

- What JARs are on the classpath
- What dependencies you added
- What beans already exist

And then it **decides what to create automatically**.

This is called **Auto-Configuration**.

If Spring MVC is on the classpath →

Spring Boot creates:

- DispatcherServlet
- JSON converters
- Default error handlers

If Hibernate is on the classpath →

Spring Boot creates:

- DataSource
- EntityManagerFactory
- TransactionManager

If no DataSource is defined →

Spring Boot creates a default one.

But if YOU define one →

Spring Boot backs off.

This is how Spring Boot is **smart**.

It uses conditions like:

- @ConditionalOnClass
- @ConditionalOnMissingBean

This is how it knows when to auto-configure and when not to.

What Starters Really Do

Spring Boot starters are not just dependency bundles.

They encode:

- Which libraries belong together
- Which versions are compatible
- Which auto-configurations should trigger

Instead of manually finding:

- Spring MVC
- Jackson
- Validation
- Tomcat
- Logging

You just write:

`spring-boot-starter-web`

Boot does the rest.

This removed dependency hell from Java backend development.

Spring Boot Is Not a Replacement — It Is the Evolution

Spring = Enterprise Architecture Framework

Spring Boot = Application Platform on top of Spring

Spring gives you:

- DI
- IoC
- MVC
- ORM
- AOP

Spring Boot gives you:

- Auto-configuration
- Starters
- Embedded servers
- Opinionated defaults
- Zero-XML setup

Spring Boot did not change what Spring does.

It changed how **fast** and **safely** you can use Spring.

That is why today:

Every modern Spring project is a Spring Boot project.

Because in enterprise systems:

Architecture matters.

But **velocity** matters even more.